

Phase 5 Test Specifications

POC-07 — Inventory Management & Procurement System

Total Test Cases: 25 | **Pass Threshold:** 18 of 25 (70%)

Test Fixtures

```
import pytest
from unittest.mock import MagicMock, patch
from multi_agent.state import InventoryAnalysisState, initial_state

@pytest.fixture
def sample_product_data():
    return {
        "id": 1, "sku": "SKU-GRO-0001", "name": "Basmati Rice 5kg",
        "category": "grocery", "unit_price": 350.0, "cost_price": 280.0,
        "reorder_point": 20, "reorder_quantity": 100, "supplier_id": 1,
        "stock_level": {"quantity_on_hand": 12, "quantity_available": 12,
        "quantity_reserved": 0}
    }

@pytest.fixture
def state_with_data(sample_product_data):
    return {**initial_state(1), "product_data": sample_product_data}

@pytest.fixture
def state_after_forecast(state_with_data):
    return {**state_with_data, "demand_forecast": {
        "avg_daily_demand": 4.0, "demand_trend": "stable",
        "days_of_stock_remaining": 3, "stockout_risk": "high",
        "forecast_notes": "Will stock out in 3 days at current rate."}}

@pytest.fixture
def state_after_reorder(state_after_forecast):
    return {**state_after_forecast, "reorder_recommendation": {
        "reorder_required": True, "recommended_quantity": 100,
        "urgency": "within_3_days", "reason": "Stockout risk high."},
        "analysis_status": "reorder_required"}
```

STATE SCHEMA TESTS (4 cases)

TC-07-P5-STATE-01: TypedDict Has All Fields

```
def test_state_fields():
    import typing
```

```

    hints = typing.get_type_hints(InventoryAnalysisState)
    for f in ["product_id", "product_data", "demand_forecast",
"reorder_recommendation",
            "supplier_quote", "audit_report", "analysis_status", "errors",
"messages"]:
        assert f in hints

```

TC-07-P5-STATE-02: initial_state Correct Defaults

```

def test_initial_defaults():
    s = initial_state(42)
    assert s["product_id"] == 42
    assert s["product_data"] == {}
    assert s["demand_forecast"] == {}
    assert s["reorder_recommendation"] == {}
    assert s["supplier_quote"] == {}
    assert s["audit_report"] == ""
    assert s["analysis_status"] == "analyzing"
    assert s["errors"] == [] and s["messages"] == []

```

TC-07-P5-STATE-03: No In-Place Mutation

```

def test_no_mutation(state_with_data, sample_product_data):
    orig = state_with_data["messages"].copy()
    with patch("multi_agent.agents.requests.get") as mg, \
         patch("multi_agent.agents._llm") as ml:
        mg.return_value.status_code = 200
        mg.return_value.json.return_value = sample_product_data
        ml.return_value.invoke.return_value = MagicMock(

content='{ "avg_daily_demand":4.0,"demand_trend":"stable","days_of_stock_remaining"
:3,"stockout_risk":"high","forecast_notes":"Low stock."}')
        from multi_agent.agents import demand_forecaster
        new = demand_forecaster(state_with_data)
    assert state_with_data["messages"] == orig
    assert len(new["messages"]) > len(orig)

```

TC-07-P5-STATE-04: Fields Persist Across Nodes

```

def test_fields_persist(state_after_reorder):
    assert state_after_reorder["demand_forecast"]["stockout_risk"] == "high"
    assert state_after_reorder["product_data"]["sku"] == "SKU-GRO-0001"
    assert state_after_reorder["reorder_recommendation"]["reorder_required"] is
True

```

AGENT TESTS (8 cases)

TC-07-P5-AGENT-01: Demand Forecaster Fetches Data

```
def test_demand_fetches(sample_product_data):
    s = initial_state(1)
    with patch("multi_agent.agents.requests.get") as mg,
    patch("multi_agent.agents._llm") as ml:
        mg.return_value.status_code = 200; mg.return_value.json.return_value =
        sample_product_data
        ml.return_value.invoke.return_value = MagicMock(

content={'avg_daily_demand':4.0,"demand_trend":"stable","days_of_stock_remaining"
:3,"stockout_risk":"high","forecast_notes":"Low."}')
        from multi_agent.agents import demand_forecaster
        result = demand_forecaster(s)
        mg.assert_called()
        assert result["product_data"].get("sku") == "SKU-GRO-0001"
```

TC-07-P5-AGENT-02: Demand Forecaster Handles API Error

```
def test_demand_api_error():
    s = initial_state(999)
    with patch("multi_agent.agents.requests.get", side_effect=Exception("down")),
    \
        patch("multi_agent.agents._llm") as ml:
        ml.return_value.invoke.return_value = MagicMock(

content={'avg_daily_demand':0,"demand_trend":"unknown","days_of_stock_remaining":
0,"stockout_risk":"unknown","forecast_notes":"Error."}')
        from multi_agent.agents import demand_forecaster
        result = demand_forecaster(s)
        assert len(result["errors"]) > 0
```

TC-07-P5-AGENT-03: Demand Forecast Has Required Fields

```
def test_forecast_fields(sample_product_data):
    s = initial_state(1)
    with patch("multi_agent.agents.requests.get") as mg,
    patch("multi_agent.agents._llm") as ml:
        mg.return_value.status_code = 200; mg.return_value.json.return_value =
        sample_product_data
        ml.return_value.invoke.return_value = MagicMock(

content={'avg_daily_demand':4.0,"demand_trend":"stable","days_of_stock_remaining"
:3,"stockout_risk":"high","forecast_notes":"High risk."}')
        from multi_agent.agents import demand_forecaster
        result = demand_forecaster(s)
```

```
for f in ["avg_daily_demand", "demand_trend", "stockout_risk"]:
    assert f in result["demand_forecast"]
```

TC-07-P5-AGENT-04: Reorder Agent Recommends for High Risk

```
def test_reorder_recommends(state_after_forecast):
    with patch("multi_agent.agents._llm") as ml:
        ml.return_value.invoke.return_value = MagicMock(
            content='{"reorder_required":true,"recommended_quantity":100,"urgency":"within_3_days","reason":"Stockout risk high."}')
        from multi_agent.agents import reorder_agent
        result = reorder_agent(state_after_forecast)
        assert result["reorder_recommendation"]["reorder_required"] is True
        assert result["analysis_status"] == "reorder_required"
```

TC-07-P5-AGENT-05: Reorder Agent No Action for Low Risk

```
def test_reorder_no_action():
    s = {**initial_state(1), "product_data": {"sku": "SKU-ELC-0001"},
        "demand_forecast": {"avg_daily_demand": 1.0, "stockout_risk": "none",
            "days_of_stock_remaining": 60, "demand_trend":
            "stable"}}
    with patch("multi_agent.agents._llm") as ml:
        ml.return_value.invoke.return_value = MagicMock(
            content='{"reorder_required":false,"recommended_quantity":0,"urgency":"not_required","reason":"Ample stock."}')
        from multi_agent.agents import reorder_agent
        result = reorder_agent(s)
        assert result["reorder_recommendation"]["reorder_required"] is False
```

TC-07-P5-AGENT-06: Supplier Coordinator Generates Quote

```
def test_supplier_quote(state_after_reorder):
    catalog = [{"sku": "SKU-GRO-0001", "name": "Basmati Rice", "cost_price": 280.0}]
    with patch("multi_agent.agents.requests.get") as mg,
        patch("multi_agent.agents._llm") as ml:
        mg.return_value.status_code = 200; mg.return_value.json.return_value = catalog
        ml.return_value.invoke.return_value = MagicMock(
            content='{"supplier_id":1,"quoted_unit_cost":280.0,"total_order_cost":28000.0,"estimated_lead_time_days":7,"quote_notes":"Standard rate."}')
        from multi_agent.agents import supplier_coordinator
```

```
result = supplier_coordinator(state_after_reorder)
assert result["supplier_quote"]["total_order_cost"] > 0
```

TC-07-P5-AGENT-07: Inventory Auditor Generates Report

```
def test_auditor_generates(state_after_reorder):
    with patch("multi_agent.agents._llm") as ml:
        ml.return_value.invoke.return_value = MagicMock(
            content="SKU-GRO-0001 Basmati Rice has 12 units available against a
reorder point of 20. Stockout expected in 3 days. Reorder of 100 units from
supplier 1 is recommended at ₹28,000 total.")
        from multi_agent.agents import inventory_auditor
        result = inventory_auditor(state_after_reorder)
        assert len(result["audit_report"]) > 50
        assert result["analysis_status"] == "complete"
```

TC-07-P5-AGENT-08: All Agents Append Messages

```
def test_all_append_messages(state_with_data, sample_product_data):
    initial_count = len(state_with_data["messages"])
    with patch("multi_agent.agents.requests.get") as mg,
        patch("multi_agent.agents._llm") as ml:
        mg.return_value.status_code = 200; mg.return_value.json.return_value =
sample_product_data
        ml.return_value.invoke.return_value = MagicMock(
            content='{"avg_daily_demand":4.0,"demand_trend":"stable","days_of_stock_remaining"
:3,"stockout_risk":"high","forecast_notes":"High risk."}')
        from multi_agent.agents import demand_forecaster
        result = demand_forecaster(state_with_data)
        assert len(result["messages"]) == initial_count + 1
        assert "Demand Forecaster" in result["messages"][-1]
```

ROUTING TESTS (6 cases)

TC-07-P5-ROUTE-01: Normal Routes to reorder_agent

```
def test_normal_routes_reorder():
    from multi_agent.graph import should_skip_to_audit
    s = {**initial_state(1), "analysis_status": "analyzing", "errors": []}
    assert should_skip_to_audit(s) == "reorder_agent"
```

TC-07-P5-ROUTE-02: Many Errors Routes to audit

```
def test_error_routes_audit():
    from multi_agent.graph import should_skip_to_audit
    s = {**initial_state(1), "errors": ["e1", "e2", "e3"]}
    assert should_skip_to_audit(s) == "inventory_auditor"
```

TC-07-P5-ROUTE-03: Error Status Routes to Audit

```
def test_error_status_routes():
    from multi_agent.graph import should_skip_to_audit
    s = {**initial_state(1), "errors": []}
    s["analysis_status"] = "error"
    assert should_skip_to_audit(s) == "inventory_auditor"
```

TC-07-P5-ROUTE-04: Returns Valid Node Name

```
def test_valid_node():
    from multi_agent.graph import should_skip_to_audit
    valid = {"reorder_agent", "inventory_auditor"}
    for n in [0, 1, 2, 3, 5]:
        s = {**initial_state(1), "errors": [f"e{i}" for i in range(n)]}
        assert should_skip_to_audit(s) in valid
```

TC-07-P5-ROUTE-05: Graph Has 4 Nodes

```
def test_4_nodes():
    from multi_agent.graph import build_inventory_graph
    import inspect
    src = inspect.getsource(build_inventory_graph)
    for n in ["demand_forecaster", "reorder_agent", "supplier_coordinator",
              "inventory_auditor"]:
        assert n in src
```

TC-07-P5-ROUTE-06: Entry Point Is demand_forecaster

```
def test_entry_point():
    from multi_agent.graph import build_inventory_graph
    import inspect
    src = inspect.getsource(build_inventory_graph)
    assert "demand_forecaster" in src and ("set_entry_point" in src or
      "entry_point" in src.lower())
```

END-TO-END TESTS (7 cases)

TC-07-P5-E2E-01: Full Pipeline Executes

```
def test_full_pipeline(sample_product_data):
    resps = [

        '{"avg_daily_demand":4.0,"demand_trend":"stable","days_of_stock_remaining":3,"stockout_risk":"high","forecast_notes":"Low."}',

        '{"reorder_required":true,"recommended_quantity":100,"urgency":"within_3_days","reason":"Stockout risk."}',

        '{"supplier_id":1,"quoted_unit_cost":280.0,"total_order_cost":28000.0,"estimated_lead_time_days":7,"quote_notes":"Standard."}',
        "SKU-GRO-0001 needs reorder urgently. Place PO for 100 units at ₹28,000."
    ]
    cnt = [0]
    def side(p): r = MagicMock(); r.content = resps[min(cnt[0], 3)]; cnt[0] += 1;
    return r
    with patch("multi_agent.agents.requests.get") as mg,
    patch("multi_agent.agents._llm") as ml:
        mg.return_value.status_code = 200; mg.return_value.json.return_value = sample_product_data
        ml.return_value.invoke.side_effect = side
        from multi_agent.graph import analyze_product
        try:
            result = analyze_product(product_id=1)
            assert result is not None
        except Exception as e:
            pytest.fail(f"analyze_product raised: {e}")
```

TC-07-P5-E2E-02: audit_report Non-Empty

```
def test_audit_non_empty(sample_product_data):
    resps = [

        '{"avg_daily_demand":2.0,"demand_trend":"stable","days_of_stock_remaining":10,"stockout_risk":"low","forecast_notes":"OK."}',

        '{"reorder_required":false,"recommended_quantity":0,"urgency":"not_required","reason":"Sufficient."}',

        '{"supplier_id":null,"quoted_unit_cost":0,"total_order_cost":0,"estimated_lead_time_days":7,"quote_notes":"No quote needed."}',
        "Stock levels are adequate. No immediate action required."
    ]
    cnt = [0]
    def side(p): r = MagicMock(); r.content = resps[min(cnt[0], 3)]; cnt[0] += 1;
    return r
```

```

    with patch("multi_agent.agents.requests.get") as mg,
    patch("multi_agent.agents._llm") as ml:
        mg.return_value.status_code = 200; mg.return_value.json.return_value =
sample_product_data
        ml.return_value.invoke.side_effect = side
        from multi_agent.graph import analyze_product
        result = analyze_product(product_id=1)
        assert len(result.get("audit_report", "")) > 20

```

TC-07-P5-E2E-03: Four Messages in Trail

```

def test_four_messages(sample_product_data):
    resps = [

'{"avg_daily_demand":2.0,"demand_trend":"stable","days_of_stock_remaining":10,"sto
ckout_risk":"low","forecast_notes":"OK."}',

'{"reorder_required":false,"recommended_quantity":0,"urgency":"not_required","reas
on":"OK."}',

'{"supplier_id":null,"quoted_unit_cost":0,"total_order_cost":0,"estimated_lead_tim
e_days":7,"quote_notes":"OK."}',
        "Inventory healthy."
    ]
    cnt = [0]
    def side(p): r = MagicMock(); r.content = resps[min(cnt[0], 3)]; cnt[0] += 1;
    return r
    with patch("multi_agent.agents.requests.get") as mg,
    patch("multi_agent.agents._llm") as ml:
        mg.return_value.status_code = 200; mg.return_value.json.return_value =
sample_product_data
        ml.return_value.invoke.side_effect = side
        from multi_agent.graph import analyze_product
        result = analyze_product(product_id=1)
        assert len(result.get("messages", [])) >= 4

```

TC-07-P5-E2E-04: Status Is Terminal

```

def test_terminal_status(sample_product_data):
    resps = [

'{"avg_daily_demand":2.0,"demand_trend":"stable","days_of_stock_remaining":10,"sto
ckout_risk":"low","forecast_notes":"OK."}',

'{"reorder_required":false,"recommended_quantity":0,"urgency":"not_required","reas
on":"OK."}',

'{"supplier_id":null,"quoted_unit_cost":0,"total_order_cost":0,"estimated_lead_tim
e_days":7,"quote_notes":"OK."}',

```



```

        "Normal."
    ]
    cnt = [0]
    def side(p): r = MagicMock(); r.content = resps[min(cnt[0], 3)]; cnt[0] += 1;
    return r
    with patch("multi_agent.agents.requests.get") as mg,
    patch("multi_agent.agents._llm") as ml:
        mg.return_value.status_code = 200; mg.return_value.json.return_value =
sample_product_data
        ml.return_value.invoke.side_effect = side
        from multi_agent.graph import analyze_product
        result = analyze_product(product_id=1)
        assert result["analysis_status"] in ("complete", "reorder_required",
"healthy", "analyzing")

```

TC-07-P5-E2E-05: LangSmith Traces

```

def test_langsmith():
    import os
    if not os.getenv("LANGCHAIN_API_KEY"): pytest.skip("No key")
    from langsmith import Client
    runs = list(Client().list_runs(project_name="AI-Readiness-POC-07-P5",
limit=5))
    if not runs: pytest.skip("No traces yet")
    assert len(runs) >= 1

```

TC-07-P5-E2E-06: API Failure Handled

```

def test_api_failure():
    resps = [

'{"avg_daily_demand":0,"demand_trend":"unknown","days_of_stock_remaining":0,"stock
out_risk":"unknown","forecast_notes":"API error."}',

'{"reorder_required":false,"recommended_quantity":0,"urgency":"not_required","reas
on":"Insufficient data."}',

'{"supplier_id":null,"quoted_unit_cost":0,"total_order_cost":0,"estimated_lead_tim
e_days":7,"quote_notes":"No data."}',
        "Unable to retrieve product data. Please verify product ID and API
connectivity."
    ]
    cnt = [0]
    def side(p): r = MagicMock(); r.content = resps[min(cnt[0], 3)]; cnt[0] += 1;
    return r
    with patch("multi_agent.agents.requests.get", side_effect=Exception("API
down")), \
        patch("multi_agent.agents._llm") as ml:
        ml.return_value.invoke.side_effect = side

```

```

from multi_agent.graph import analyze_product
try:
    result = analyze_product(product_id=1)
    assert result is not None and len(result.get("errors", [])) > 0
except Exception as e:
    pytest.fail(f"Should not crash: {e}")

```

TC-07-P5-E2E-07: Urgent Reorder Sets Status

```

def test_urgent_reorder(sample_product_data):
    resps = [

        '{"avg_daily_demand":8.0,"demand_trend":"increasing","days_of_stock_remaining":1,"stockout_risk":"high","forecast_notes":"Critical."}',

        '{"reorder_required":true,"recommended_quantity":200,"urgency":"immediate","reason":"Stockout tomorrow."}',

        '{"supplier_id":1,"quoted_unit_cost":280.0,"total_order_cost":56000.0,"estimated_lead_time_days":5,"quote_notes":"Urgent order."}',
        "URGENT: SKU-GRO-0001 will stock out tomorrow. Place immediate PO for 200 units."
    ]
    cnt = [0]
    def side(p): r = MagicMock(); r.content = resps[min(cnt[0], 3)]; cnt[0] += 1;
    return r
    with patch("multi_agent.agents.requests.get") as mg,
    patch("multi_agent.agents._llm") as ml:
        mg.return_value.status_code = 200; mg.return_value.json.return_value = sample_product_data
        ml.return_value.invoke.side_effect = side
        from multi_agent.graph import analyze_product
        result = analyze_product(product_id=1)
        if result.get("reorder_recommendation", {}).get("reorder_required"):
            assert result["analysis_status"] in ("reorder_required", "complete")

```